

莫队算法

前面我们学习过莫队算法，莫队算法号称可以解决一切可以离线的序列问题，它的本质就是一种巧妙的暴力，如果暴力可以做，并且支持离线查询，那么莫队就一定可以做。

所以它相对于数据结构，动态规划等算法在解决序列问题的时候用途更加广泛，限制更少。

特别是一些比较复杂的问题，以我们现在学习过的算法和数据结构，很难直接就套用上去，特别是现在的比赛题天马行空，很多题我们平时练习的时候几乎没有接触过，暴力，特别是一种巧妙的暴力是很有用的办法。

莫队算法

普通的序列问题我们已经学会了如何用莫队算法求解，带一些简单修改的莫队我们也学习过带修改的莫队，但是在比赛中更多的遇到的是树上问题，那么对于树上的问题，或者说树上的序列问题，比如树上两点之间的路径也就相当于序列，但是这种序列和我们普通的序列不一样。

那么对于树上的序列问题，特别是树上非合并的序列问题，我们该如何用莫队算法来求解呢？

树上莫队

首先，很容易想到，树上任意两点间的路径有重叠部分也有独立的部分，他和我们普通的序列不太一样，所以就和我们以前处理树的问题一样，我们一般都把树变成链来处理，这条链就是我们需要的区间。

树变链的办法有很多种，常见的比如树链剖分，dfs序，bfs序等等。但是我们一定要保证这种树变链的办法可以像普通的序列莫队那样可以左右端点移动。比如上一个区间是 $[l1, r1]$ ，下一次区间变为 $[l2, r2]$ ，挪动左右端点时，表示的区间是正确的。

很明显，这三种树变链的方式都不满足这个条件。

树上莫队

我们以dfs序为例：

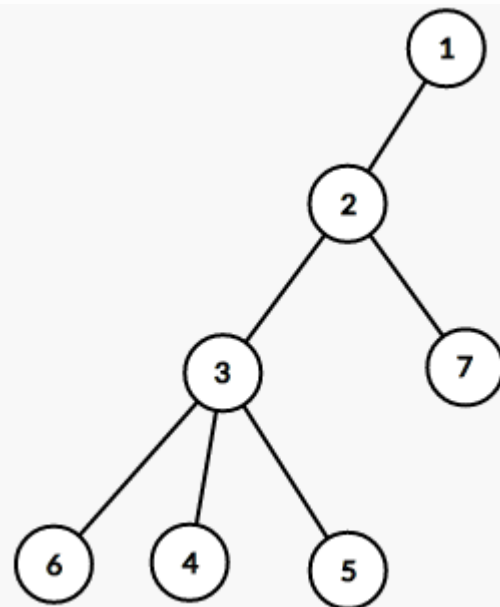
下图的dfs序为：1 2 3 6 4 5 7

如果我要求 $2 \rightarrow 4$ 的路径上出现的数的种类

但是我们会发现dfs序中，2到4中，出现一些在 $2 \rightarrow 4$ 这条链中未出现的点，如点6。

同样我们再看 $2 \rightarrow 7$ ，在树上这条链中只有两个点，但是在dfs中却有一大堆无用的点。

所以通过dfs序来把树变成序列的做法错误的，同样，bfs序和树链剖分的方法都会出现这种问题



树上莫队

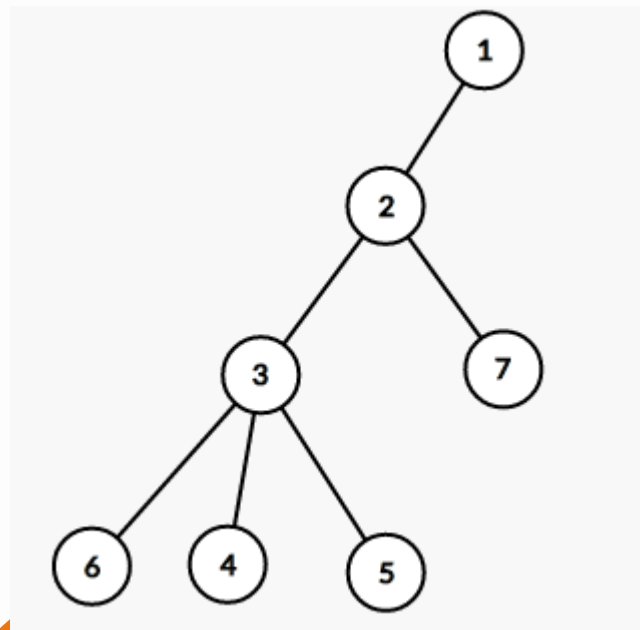
以前的变树为链的做法都不满足条件，这里介绍一种类似于dfs序的一种树变链的做法——欧拉序。欧拉序和dfs其实是一样的，只是欧拉序在记录的时候进入这个点的时候记录了一次，出去这个点的时候也要记录一次，而dfs只是记录了进入该点的时间。

dfs序：

1 2 3 6 4 5 7

欧拉序：

1 2 3 6 6 4 4 5 5 3 7 7 2 1



树上莫队

欧拉序： 1 2 3 6 6 4 4 5 5 3 7 7 2 1

可能看上去欧拉序和dfs差不多，不过就是序列中每个点出现了两次，但是我们仔细看一下：

(1) $2 \rightarrow 6$

欧拉序： 2 3 6

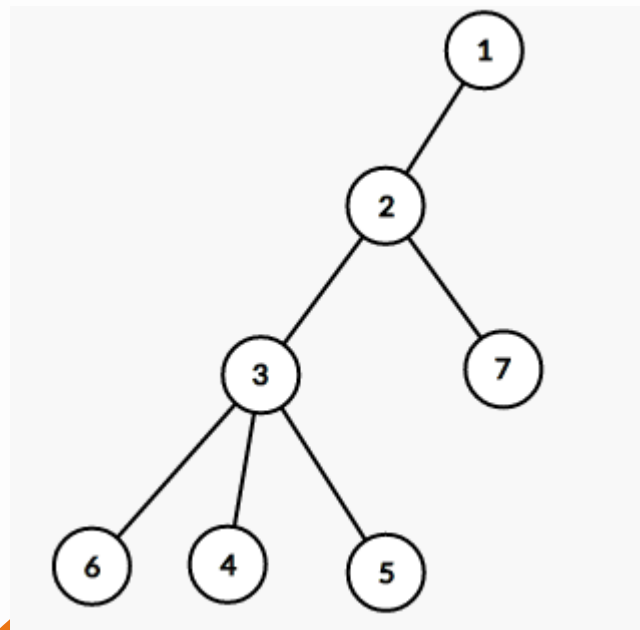
(2) $2 \rightarrow 5$

欧拉序： 2 3 6 6 4 4 5

(3) $2 \rightarrow 7$

欧拉序： 2 3 6 6 4 4 5 5 3 7

虽然在欧拉序中也会出现一些无用的点，但是我们仔细观察就会发现这些的点都出现了两次，有用的点都只出现了一次。



树上莫队

所以我们如果通过欧拉序就可以成功的把树上问题转换成区间问题，并且可以通过普通的莫队算法来实现。

但是这里还有点小的缺陷，比如 $4 \rightarrow 5$ ：

欧拉序：4 5

以及 $3 \rightarrow 7$

欧拉序：3 7

我们发现，如果两点没有祖先关系，即一个点时另一个点的祖先，那么使用欧拉序的时候就会出现这个问题，但是这个问题都是一样的——没有计算他们的最近公共祖先，所以我们只需要先预处理出来他们的最近公共祖先就可以了，最后再判断一下，看下是否需要把这个点加入进来。

Count on a tree II

题意翻译

- 给定 n 个结点的树，每个结点有一种颜色。
- m 次询问，每次询问给出 u, v ，回答 u, v 之间的路径上的结点的不同颜色数。
- $1 \leq n \leq 4 \times 10^4$, $1 \leq m \leq 10^5$ 。

输入输出样例

输入 #1

复制

```
8 2
105 2 9 3 8 5 7 7
1 2
1 3
1 4
3 5
3 6
3 7
4 8
2 5
7 8
```

输出 #1

复制

```
4
4
```

树上莫队

该题就是一道典型的树上莫队的入门题。现在我们先了解了树上莫队的大致步骤：

(1) 树变链，求出一棵树的欧拉序

(2) 预处理出来所有询问的最近公共祖先，可以用树剖，也可以用树上倍增，因为该题可以离线，所以也可以用tarjain求lca，效率更高，但是该题瓶颈在莫队，所以 $n\log n$ 的算法就完全可以过。

(3) 对树分块，分块的办法参考王室联邦分块。

(4) 裸的普通莫队处理询问

树上莫队

树上欧拉序

```
int in[maxn],out[maxn];//记录时间戳
int cnt;
void dfs(int u,int fa)
{
    in[u]=++cnt;//进入时记录时间
    for(int i=head[u];i;i=edge[i].next)
    {
        int v=edge[i].to;
        if(v!=fa)
            dfs(v,u);
    }
    out[u]=++cnt;//出去时记录时间
}
```



树上莫队

求最近公共祖先

略





树上莫队

王室联邦分块，按照 $\sqrt{n}$ 分块

略

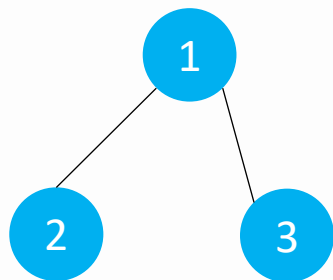


树上莫队

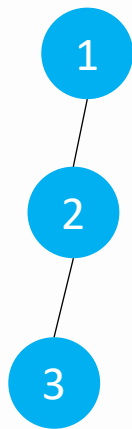
按照裸的序列莫队进行操作.

```
while(l<in[q[i].u])
{ // 既要判断颜色出现次数, 也要判断点出现次数
    flag[q[i].u]--; // 记录该点出现的次数
    if(flag[q[i].u]==1) // 只剩下一个点, 需要加点
        book[a[q[i].u]]++;
    else // 否则需要删点
        book[a[q[i].u]]--;
    if(book[a[q[i].u]]==0) // 如果这种颜色没有了
        numb--;
    else if(book[a[q[i].u]]==1) // 刚好有了一个
        numb++;
}
```

树上莫队



欧拉序: 1 2 2 3 3 1



欧拉序: 1 2 3 3 2 1

在计算 $2 \rightarrow 3$ 的时候，如果要答案正确，那么就要计算第第二个2和第一个3，即走出2的时间和进入3的时间。

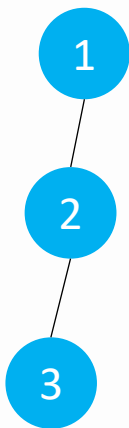
在计算 $2 \rightarrow 3$ 的时候，如果要答案正确，那么就要计算第第一个2和第一个3，即走进2的时间和进入3的时间。

明显计算方法是不统一，取决于2和3是否是根和子树的关系，所以我们要换一种计算方法。

树上莫队

对于树上任意两点，他们之间的关系只有两种，第一种是其中一个点是另一个点的祖先，第二种是没有关系

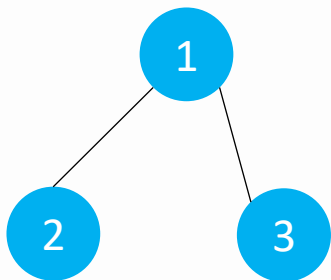
对于一个点是另一个点祖先的情况，我们可以用进入这个祖先的时间戳 $\rightarrow$ 进入子孙结点的时间戳来表示，如下图，就可以用 $2 \rightarrow 3$ 来表示，而且就算起点和终点相等也是正确的。



欧拉序: 1 2 3 3 2 1

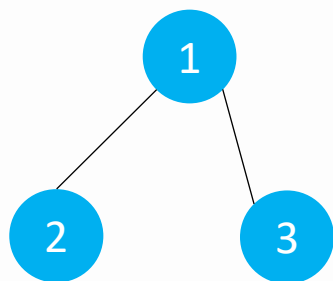
树上莫队

当两种情况没有关系的时候，我们可以用出前一个点的时间戳 $\rightarrow$ 进入该点的时间戳来表示，至于为什么，自己模拟一下。如果不满足大小关系，那么就交换位置再计算。如下图， $2 \rightarrow 3$ 来表示



欧拉序: 1 2 2 3 3 1

树上莫队



欧拉序: 1 2 2 3 3 1
颜色 1 2 2 3 3 1

最开始时 $l=1, r=1, flag[1]=1, book[1]=1, sum=1$

(1) 求3-->3的颜色种类

因为其中一个点是另一个点的祖先(相同也可以算作祖先), 那么进入点3的时间戳是4, 进入点3的时间戳也是4.

先r右移

$flag[2]=1$ 加点 $book[2]=1, sum=2;$

$flag[2]=2$ 删点 $book[2]=0, sum=1;$

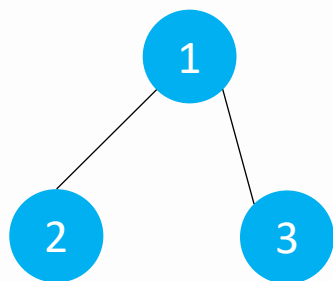
$flag[3]=1$ 加点 $book[3]=1, sum=2;$

l右移

$flag[1]=0$ 删点 $book[1]=0, sum=1$

再加入点2, 删除点2, 最后 $sum=1$ 就是答案

树上莫队



欧拉序: 1 2 2 3 3 1
颜色 1 2 2 3 3 1

此时 $l=2$ $r=2$ $sum=1$

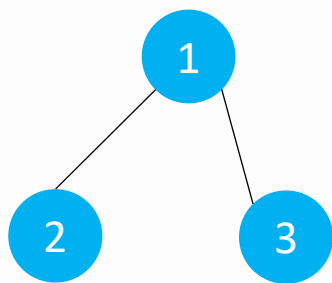
(2) 求2-->3的颜色种类

因为2和3没有父子关系，所以我们先比较 $out[2] < in[3]$ ，
 $out[3] > in[2]$ ，所以我们最终让 l 到达 $out[2]$ ， r 到达 $in[3]$
所以只需要 l 左移

$flag[2]=1$ 加点 $book[2]=1$ $sum=2$

因为两者不是祖孙关系，所以还要加上他们的最近公共祖先这个点，再判断一下点1的颜色是否出现过，所以 $sum=3$

树上莫队



欧拉序: 1 2 2 3 3 1
颜色 1 2 2 3 3 1

此时 $l=1$ $r=2$ $sum=2$

(3) 求 $1 \rightarrow 2$ 的颜色种类

因为点1是点2的祖先，所以我们考虑 $in[1] \rightarrow in[2]$

l 左移

$flag[2]=2$ 删点 $book[2]=0$, $sum=1$

$flag[1]=1$ 加点 $book[1]=1$, $sum=2$

$flag[3]=0$ 删点 $book[3]=0$, $sum=1$

$flag[2]=1$ 加点 $book[2]=1$, $sum=2$